3C

DESARROLLO DE SOFTWARE

Desarrollo de aplicaciones de Software para la solución de problemas

Saber, valorar, elegir

PADRE ARRUPE
Colegio Español

DOCENTE: Daniel Sosa

COLEGIO ESPAÑOL PADRE ARRUPE

Colegio internacional.

Tercer Año de Bachillerato de Software

Desarrollo de aplicaciones de Software para la solución de problemas

Periodo 1.

GUÍA 1 – JBDC

OBJETIVOS DE LA PRÁCTICA

Objetivo: Comprender la estructura y sintaxis de Java, manejar variables y tipos de datos, utilizar operadores y aplicar estructuras de control de flujo.

INDICACIÓN GENERAL: Introducir variables y tipos de datos, explicar operadores y expresiones, y enseñar estructuras de control de flujo

Nota: La guía forma parte integral de la clase, por lo que su finalización se revisa durante la misma.

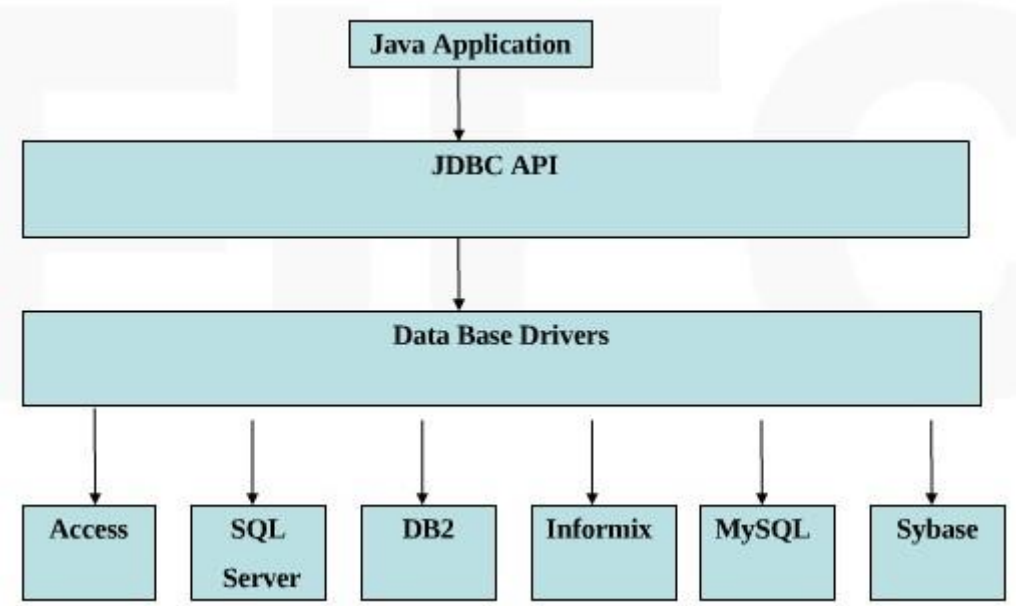
INTRODUCCIÓN TEÓRICA

Conexión a Bases de Datos con NetBeans IDE 8 o superior

JDBC (Java DataBase Connectivity) es la tecnología Java que permite a las aplicaciones interactuar directamente con motores de base de datos relacionales.

La API JDBC es una parte integral de la plataforma Java, por lo tanto, no es necesario descargar ningún paquete adicional para usarla. JDBC es una interface única que independiza a las aplicaciones del motor de la base de datos.

Un driver JDBC es usado por la JVM para introducir las invocaciones JDBC en invocaciones que la base de datos entiende.



Conectar a la base de datos

Para que una aplicación en Java se comunique con una base de datos usando la API JDBC, se requiere de un conector que realice la conexión de la aplicación a la Base de Datos.

Ese conector es específico para el manejador de base de datos y viene en la forma de un archivo ".jar" o ".zip".

Por ejemplo:

MySQL, está en el archivo: mysql-connector-java5.1.6.zip (mysql-connector-java-5.1.6-bin.jar)

SQL Server, está en el archivo: sqljdbc_1.2.2828.100_esn.zip (sqljdbc.jar)

ENLACE DE DESCARGA JDBC:
https://dev.mysql.com/downloads/connector/j/

NetBeans nos permite realizar algunas tareas relacionadas con bases de datos:

1. Conectar una aplicación a una base de datos.
2. Conectar a NetBeans directamente a una base de datos para crear, eliminar, modificar tablas, agregar, eliminar, modificar registros y hacer consultas de datos.
3. Generar aplicaciones a partir de ingeniería inversa.
4. Crear persistencia con la Base de Datos.
5. Otras innumerables.

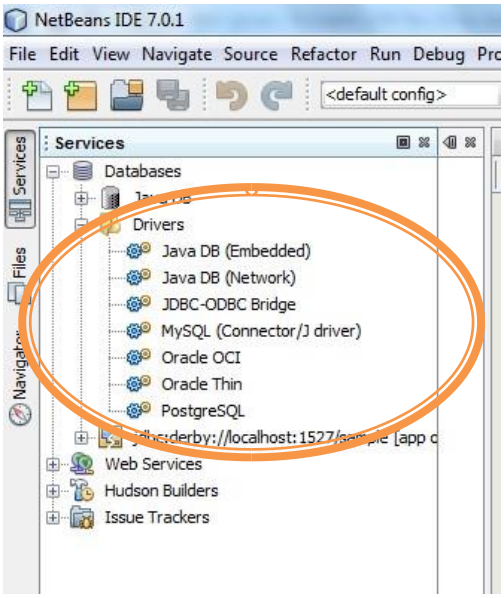
Conexión de una aplicación a una base de datos

Para conectar a una aplicación a una base de datos, se requiere:

1. Agregar a NetBeans el conector como una biblioteca o librería. Esto permite que el conector esté disponible para los proyectos.
2. Agregar a un proyecto el conector. Esto permite que la aplicación se pueda conectar a la base de datos.

Agregar a NetBeans el conector para utilizar una base de datos en MySQL.

En las versiones más recientes de NetBeans, se cuenta ya con las librerías para la mayoría de gestores de bases de datos, o al menos, los más importantes.



Esto no es una regla exclusiva, puede tener una versión un poco antigua de Netbeans (por lo que se le recomienda actualizar), pueden no funcionar los drivers que posee o simplemente no estar incluidos. Certifique que lo posee, revise en su grupo de ventanas en **Services->Drivers** que posee al menos el conector MySQL (Para esta práctica)

¿No se encuentras las librerías?

Nota: Siga este procedimiento solo si no se encuentran o no funciona la librería de MySQL.

Agregaremos a Netbeans el conector de MySQL como ejemplo. El procedimiento es el siguiente:

1. Seleccionar la opción **Tools/Libraries** de la barra de menú de **NetBeans IDE 8.0 o superior** como se muestra en la

siguiente figura:

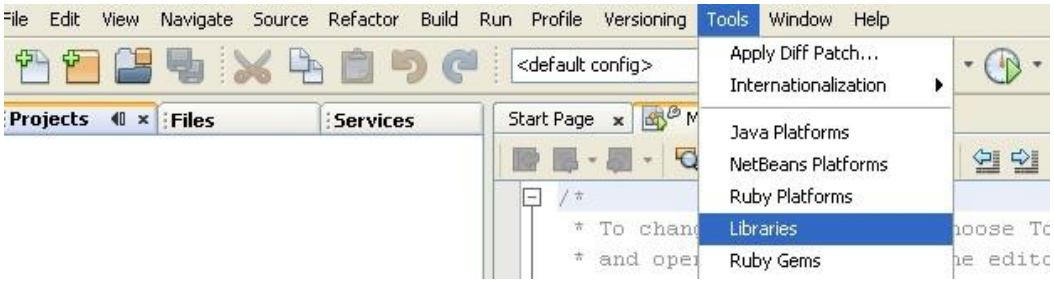


Figura 1

2. Aparece un cuadro de diálogo que nos permite administrar las bibliotecas de **NetBeans IDE 8.0.**

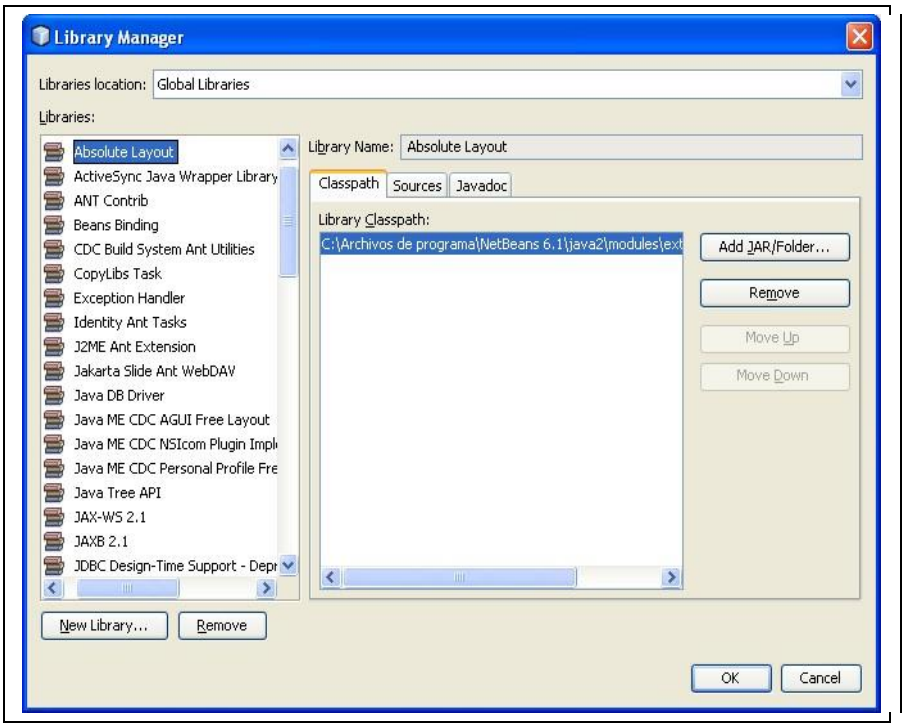


Figura 2

Del lado izquierdo del cuadro de dialogo, aparece una ventana con las bibliotecas agregadas a NetBeans. Del lado derecho aparece el nombre de la biblioteca y la trayectoria del archivo con la biblioteca. Para agregar el conector de **MySQL** a NetBeans, hacer clic en el botón **New Library…** y aparece un cuadro de dialogo como el siguiente:

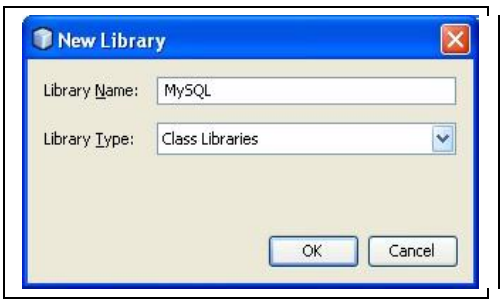


Figura 3

3. En este cuadro de dialogo se establece el nombre que tendrá el conector, digitar **MySQL** y hacer clic en el botón **OK.** Y se regresara a la ventana del administrador de bibliotecas. Hacer clic en el botón **Add JAR/Fólder…**

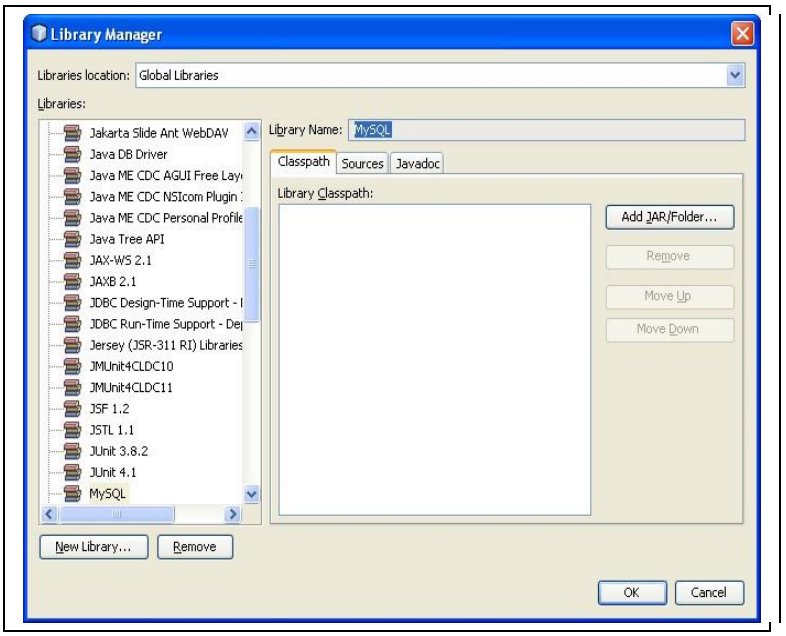


Figura 4

4. Hay que seleccionar el archivo que contiene el conector de MySQL: mysql-connector-java5.0.8-bin.jar (Puede descargalo en el sitio de oracle), después de haberlo seleccionado, hacer clic en **Add JAR/Fólder.**

Figura 5

5. Regresamos al administrador de bibliotecas en donde aparece el conector agregado, hacer clic en el botón **OK.**

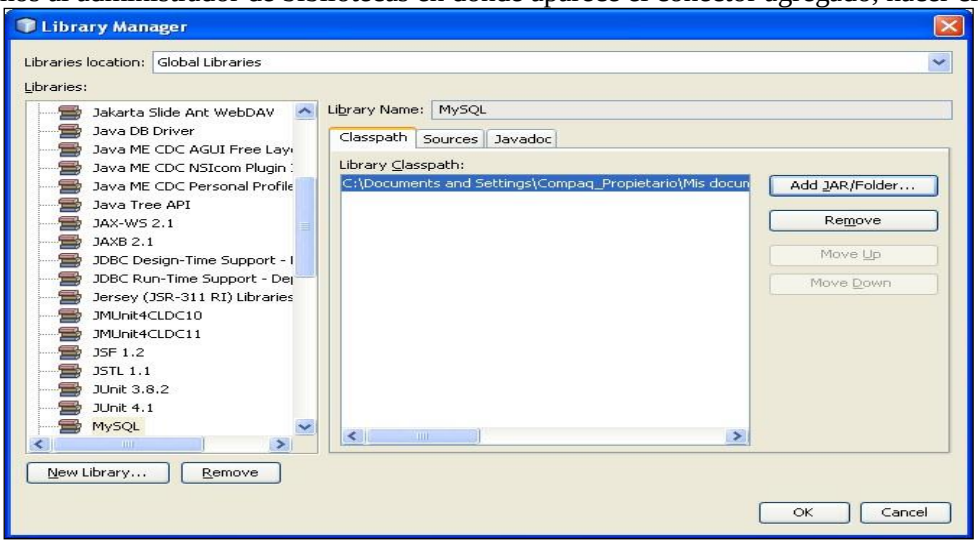


Figura 6

Nota:
El paso anterior debe seguirse para hacer la conexión de una aplicación a una Base de Datos de SQL Server también

tomar en cuenta los siguientes pasos:

- Seleccionar la opción **Tools/ Libraries** de la barra de menú como se muestra en la figura 1.
- Mostrará un cuadro de dialogo que permite administrar las bibliotecas de **NetBeans IDE 8** como se muestra en la figura 2.
- Agregar una nueva librería (**New Library**), la cual se llamará **MSSqlServer**, el nombre lo colocara en la opción **Library name** así como se muestra en la figura 3.
- Seleccionar el archivo que contiene el conector a SQL Server:

Figura 7

- Regresar al administrador de bibliotecas en donde aparece el conector agregado, así como se muestra en la figura 6, y hacer clic en el botón **OK**

Importante:

Para hacer uso de las conexiones de las bases de datos que se instalaron anteriormente, debe copiar los archivos .jar en la siguiente dirección:
C:\Archivos de programa\Java\jre1.6.0_05\lib\ext
Ó
C:\Program Files\Java\jdk1.6.0_25\jre\lib\ext

Así como se muestra en la siguiente figura:

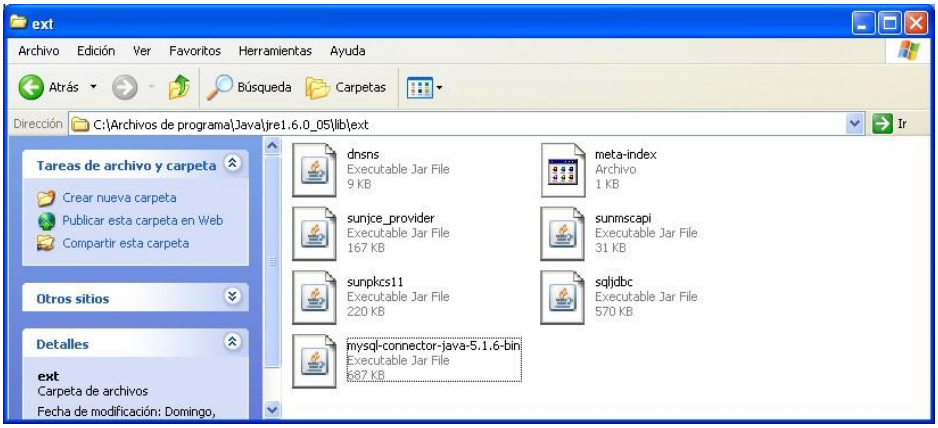


Figura 8

REALIZACIÓN DE PRÁCTICA

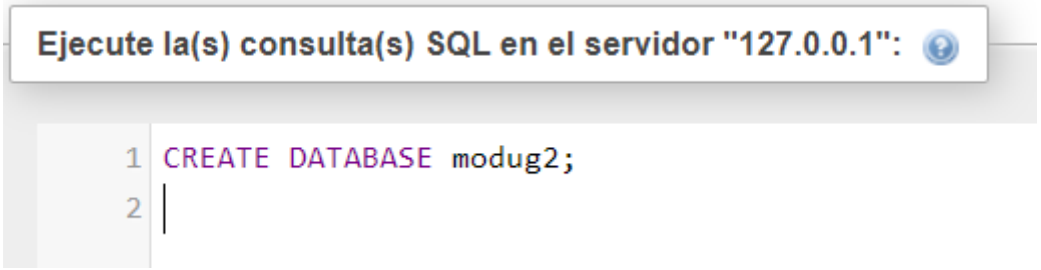
Crear un proyecto en Netbeans con el nombre ModuG2 y realizar todos los pasos que están descritos en la introducción de esta guía.

Creando una Base de datos en MySQL

Debe tener instalado en su Equipo MySQL Server o bien utilizar un gestor de BDD como PHPMyAdmin

Realizar los siguientes pasos:

1. Digitar **create database modug2** y ejecutar el query



En este paso hemos creado una Base de datos llamada Guia4, para hacer uso de la base de datos debe digitar: **use modug2 o simplemente hacer clic a la base de datos en la barra lateral izquierda y luego dar clic a la opción SQL;**

2. En este paso vamos a crear una tabla en la BD creada en el paso 1, vamos a crear Tabla Empleados con la siguiente estructura:

```
1 CREATE TABLE Empleados (
2     Codigo INT PRIMARY KEY,
3     Nombre VARCHAR(25),
4     Apellidos VARCHAR(25),
5     Telefono VARCHAR(9)
6 );
```

3. Agregar los siguientes datos a la tabla:

1	Katherine	Canales	2222-2222
2	Orquídea	Mejía	2121-2121
3	Daniel	Sosa	2324-2526

Para agregar los datos debemos utilizar transacciones SQL, en este caso se debe usar la cláusula **insert**, así como se muestra en la siguientefigura:

```
1 INSERT INTO Empleados (Codigo, Nombre, Apellidos, Telefono) VALUES
2 (1, 'Katherine', 'Canales', '2222-2222'),
3 (2, 'Orquidea', 'Mejia', '2121-2121'),
4 (3, 'Daniel', 'Sosa', '2324-2526');
```

6. Para seleccionar los datos de la tabla debe realizar una consulta **select**. En este caso se quiere seleccionar toda la información de la tabla, se debe digitar: **select * from Empleados:**

				Codigo	Nombre	Apellidos	Telefono
☐	Editar	Copiar	Borrar	1	Katherine	Canales	2222-2222
☐	Editar	Copiar	Borrar	2	Orquidea	Mejia	2121-2121
☐	Editar	Copiar	Borrar	3	Daniel	Sosa	2324-2526

Conexión de NetBeans IDE a una base de datos

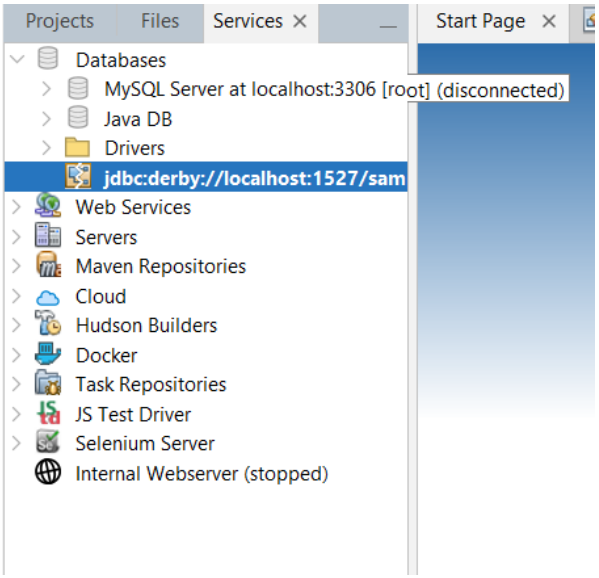
Para conectar una aplicación a una base de datos, se requiere:

- Instalar en NetBeans IDE el conector de la base de datos **SOLO SI NO LO TIENE INSTALADO**
- Establecer la conexión entre NetBeans IDE y la base de datos

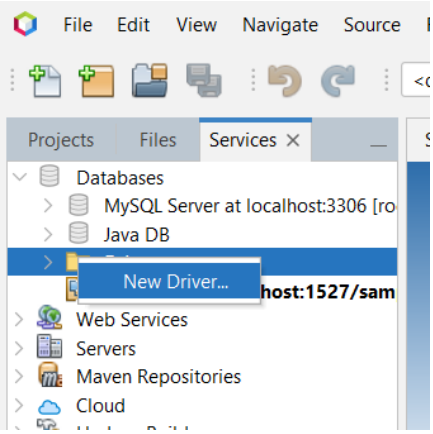
Instalación en NetBeans IDE del conector de la Base de Datos (MySQL)

El procedimiento a seguir es el siguiente:

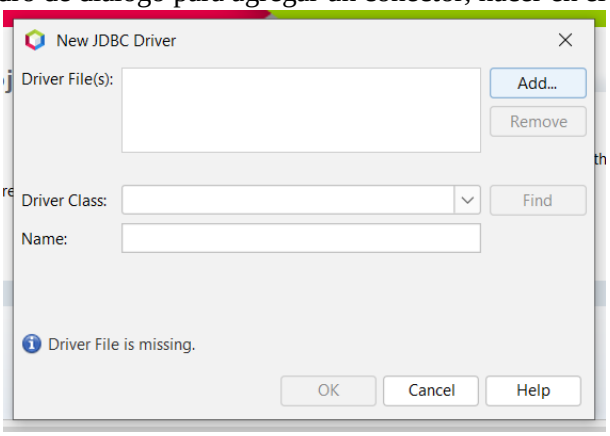
1. Hacer clic en la opción **Services** del proyecto y expanda los nodos **Databases** y **Drivers** para los conectores disponibles y las conexiones de las Base de Datos.



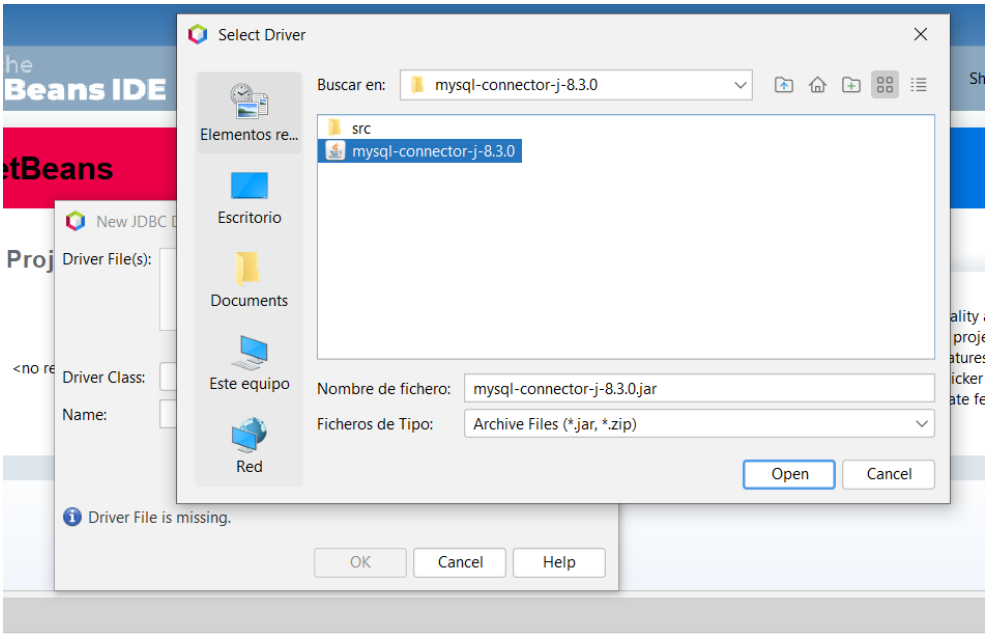
2. Para agregar el conector de MySQL, hacer clic derecho en el nodo **Drivers** y seleccione la opción **New Driver del** menú contextual.



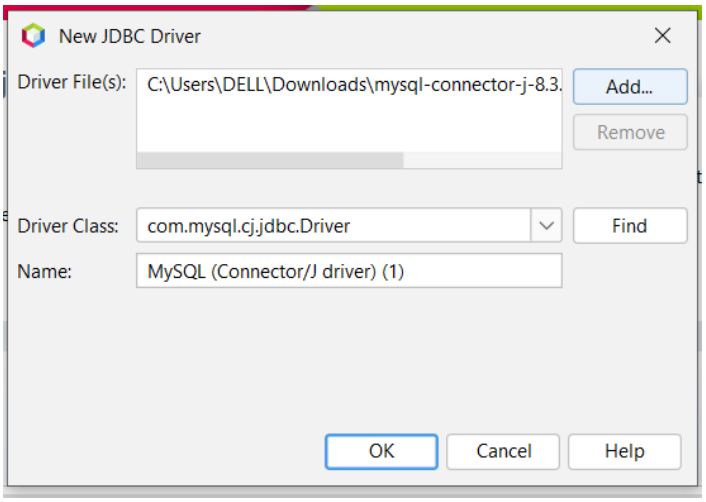
3. Aparece un cuadro de dialogo para agregar un conector, hacer en el botón **Add**



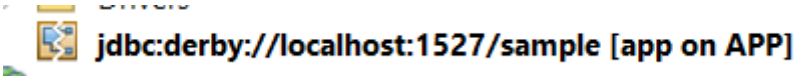
4. Seleccionar el conector, seleccionar el conector y hacer clic en el botón Abrir.



5. Aparece la información del conector seleccionado. Para confirmar hacer clic en el botón **OK.**

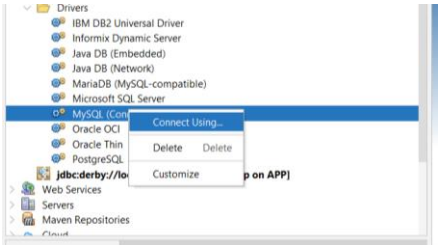


6. Se agrega un nuevo nodo para el conector de **MySQL** en la opción **Services**

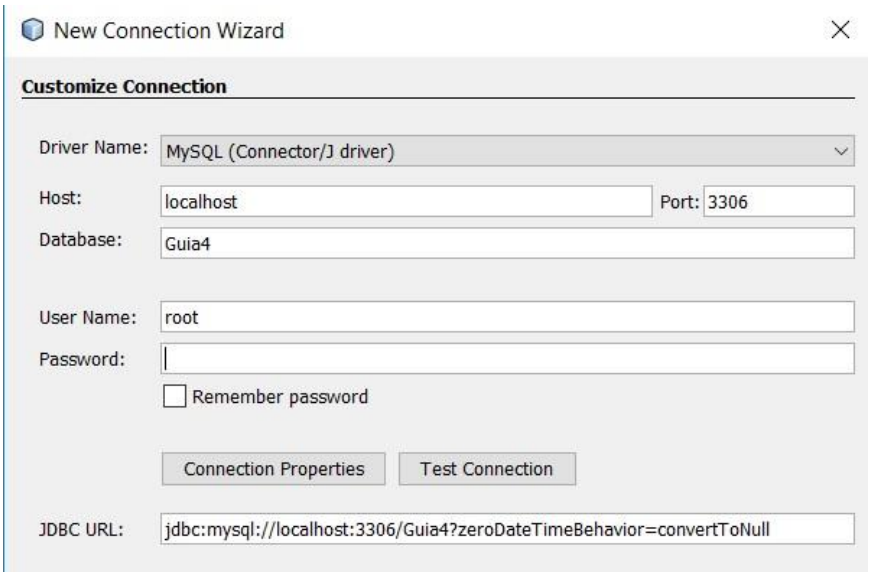


Establecer la conexión entre NetBeans IDE y la base de datos (ESTOS PASOS SÍ LOS REALIZARÁ)

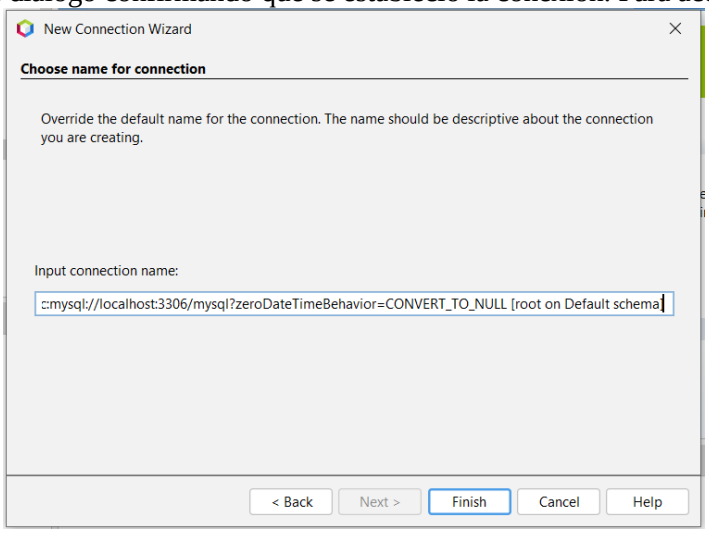
1. Hacer clic derecho sobre el nodo del conector a **MySQL**



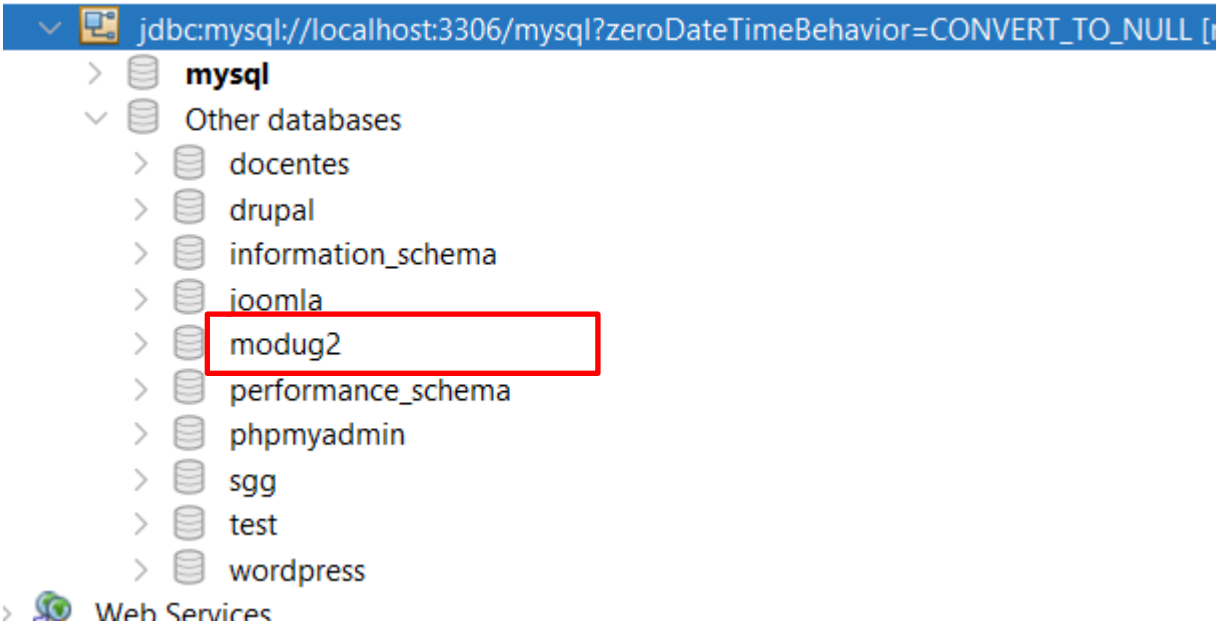
2. Seleccionar la opción **Connect…**, del menú contextual, aparece un cuadro de dialogo para establecer una nueva conexión, aparece una ventana como se muestra a continuación:
   Los datos que deben estar en esta ventana son:
   a. **En la opción Name:** MySQL (Connector/J driver)
   b. En la opción Driver: **com.mysql.jdbc.Driver**
   c. Establecer en la opcionDatabase URL el siguiente formato: jdbc:mysql://servidor:puerto/baseDatos Por ejemplo:
   jdbc:mysql://localhost:3306/mysql
   Donde **servidor** es la dirección IP (o nombre de dominio del servidor), en caso que el servidor este en la misma computadora que NetBeans utiliza el nombre localhost, **puerto**, es el puerto 3306 del servidor, si el servidor utiliza el puerto predefinido se puede omitir, **baseDatos**, es la base de datos a la que se desea conectar. Establecer el nombre del usuario
   (Username) y la contraseña (Password), para acceder a la base de datos, presionar el botón **OK.**



3. Aparece un cuadro de dialogo confirmando que se estableció la conexión. Para aceptar hacer clic en el botón **OK.**



4. Esto creará un nuevo nodo

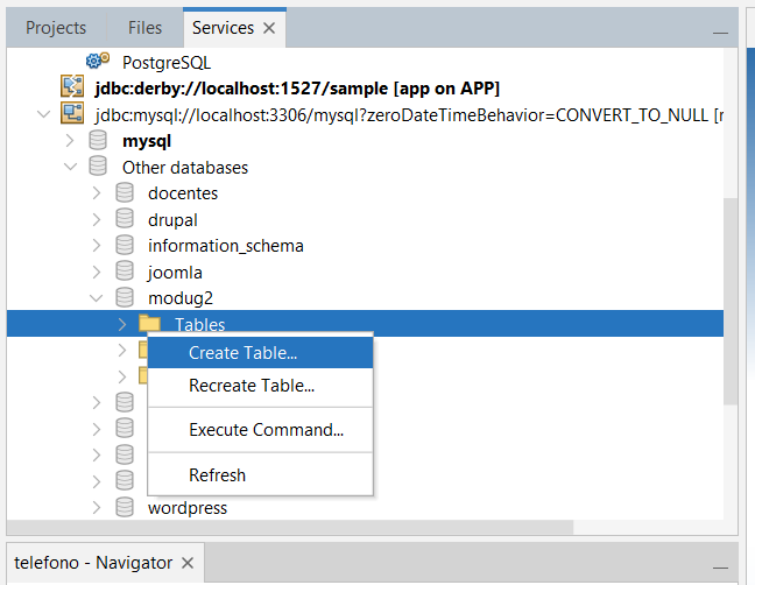


Acceso a la Base de Datos desde NetBeans IDE

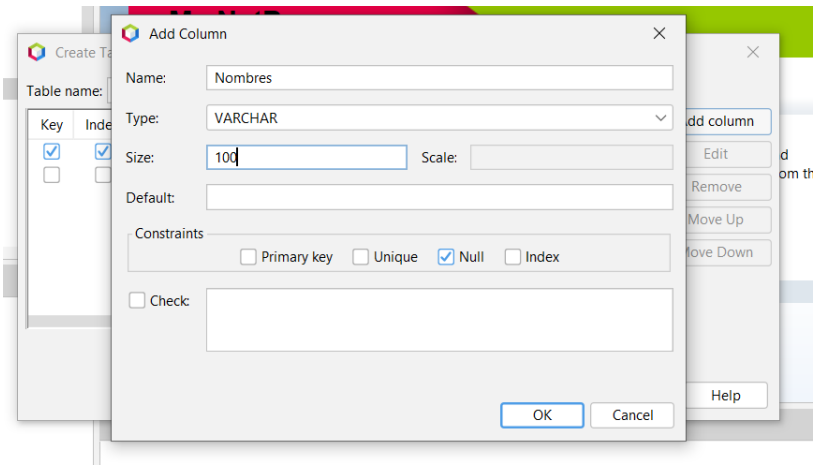
NetBeans nos permite realizar operaciones sobre la base de datos como crear y borrar tablas, agregar, eliminar, actualizar y seleccionar registros que están almacenados en las tablas, para hacer estas operaciones, debe expandir el nodo con la conexión a la base de datos para que aparezcan los objetos creados en la BD (tablas, vistas y procedimientos).

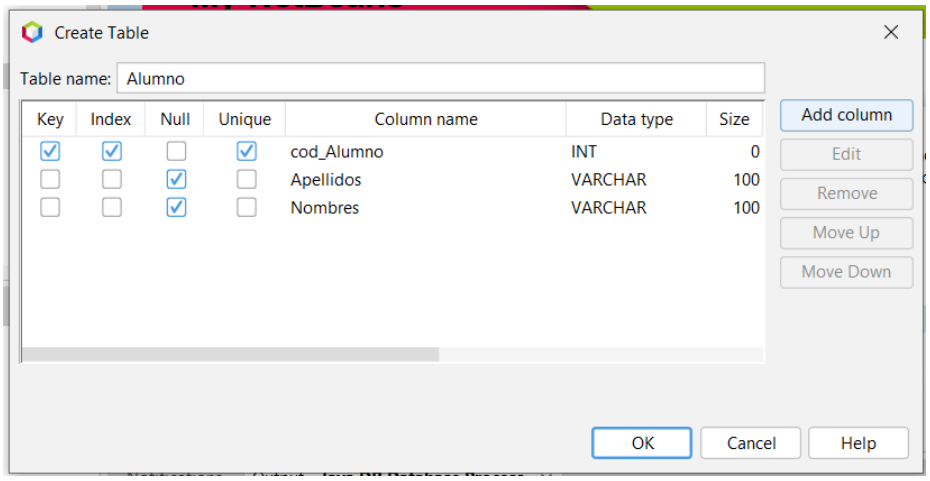
Para ver las tablas creadas en una BD, hacer clic sobre el nodo de la opción **Tables** y para ver las columnas o campos de una tabla hacer clic en (+) en el nombre de cada tabla.

Si quiere crear una tabla nueva en la BD, haga clic derecho sobre el nombre **Table** y seleccionar la opción **CreateTable…**



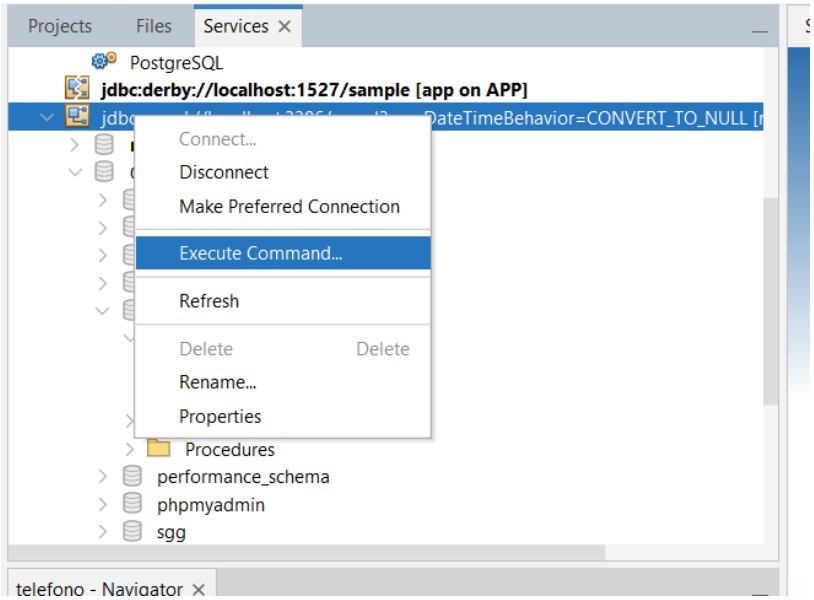
Se abre una nueva ventana donde se escribe los nombres de las columnas de la tabla, el tipo de dato, si tiene llave primaria, para agregar mas columnas hacer clic en **Addcolumn**, si quiere eliminar una columna hacer clic en **Remove.** Al terminar hacer clic en el botón **OK.**



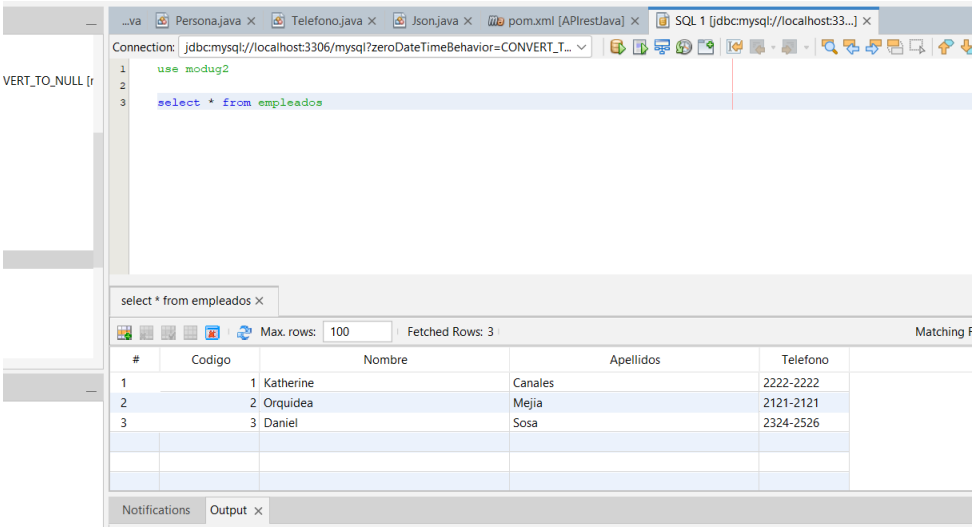


Para ejecutar un comando ya sea de inserción, eliminación, actualización y selección de información, seleccionar la opción **ExecuteCommand…**

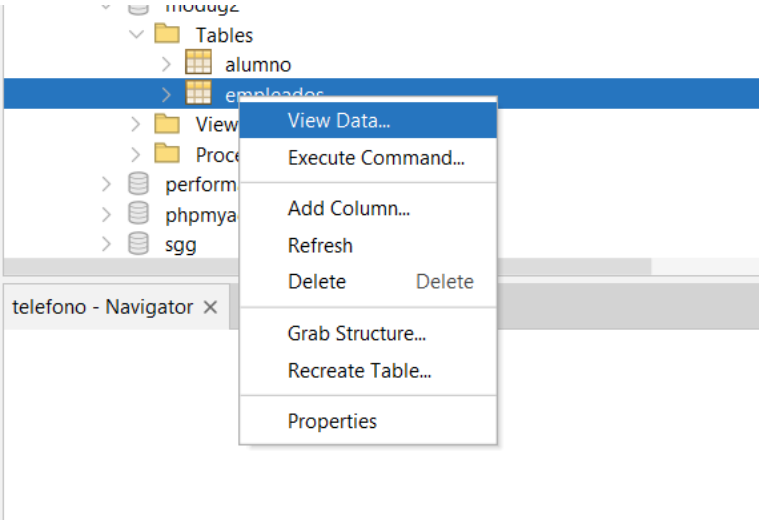
Se abrirá en Netbeans el área de edición de comandos o instrucciones SQL, donde digitamos los comandos. En la parte superior de la ventana se encuentran las transacciones y en la parte inferior muestra el resultado de la transacción. Y finalmente podemos manipular la información de una tabla



Para ejecutar la transacción y ver los resultados, se hace clic en el icono **Run SQL**.



También podemos ver los datos de una tabla seleccionando la opción **View Data,** la cual la seleccionamos haciendo clic derecho sobre el nombre de la tabla la cual queremos ver los datos.

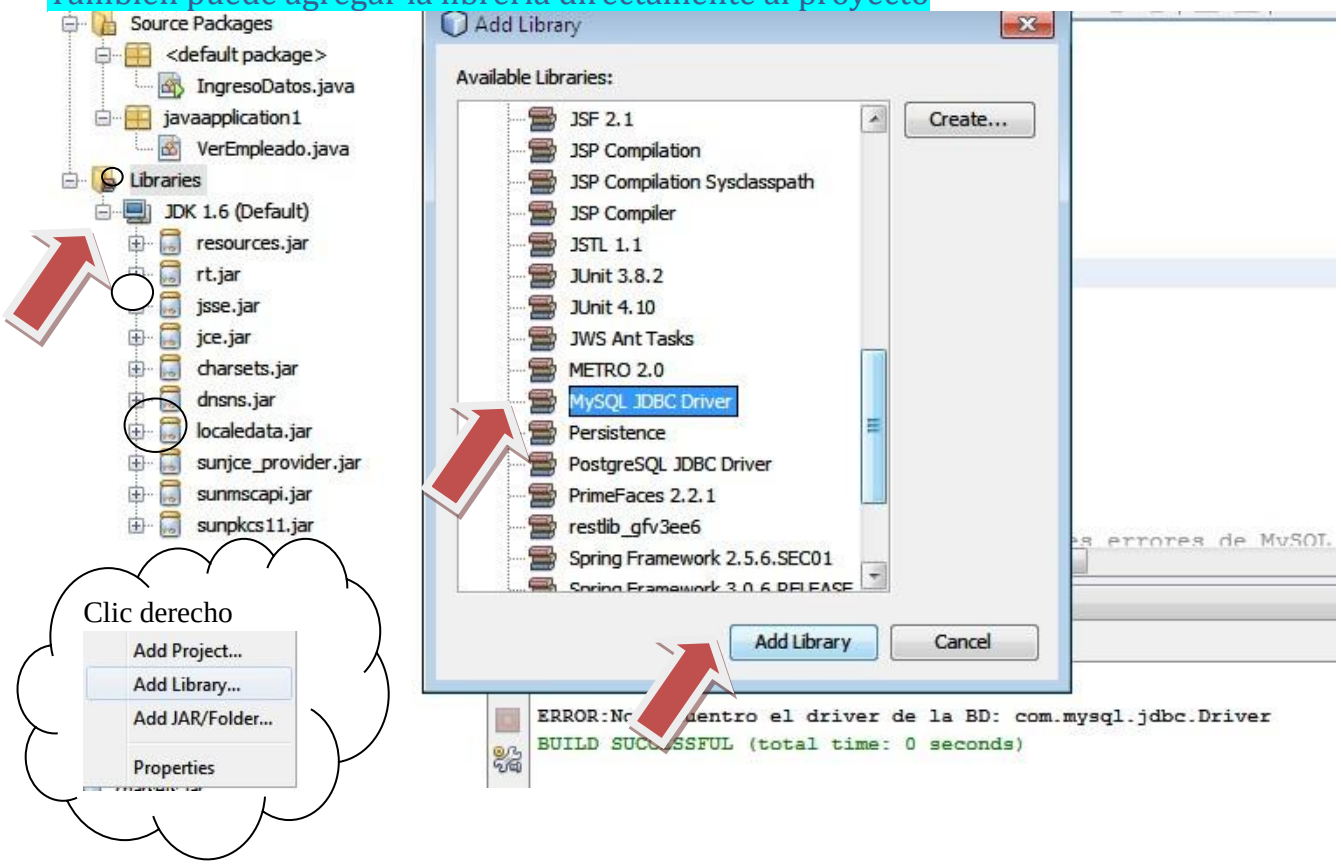


Acceder a los datos de una BD desde una aplicación Java

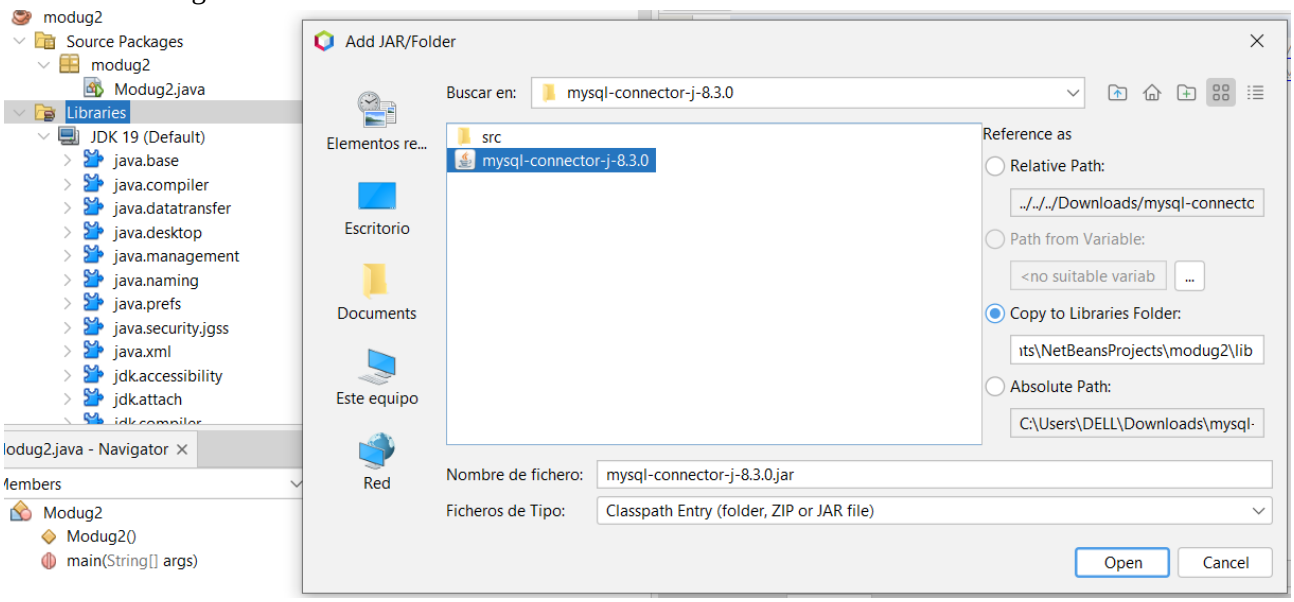
Agregue la librería de MySQL a las rutas antes mencionadas

C:\Archivos de programa\Java\jre1.6.0_05\lib\ext Ó C:\Program Files\Java\jdk1.6.0_25\jre\lib\ext

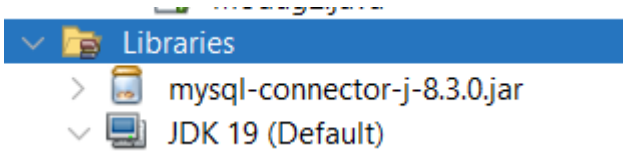
También puede agregar la librería directamente al proyecto



En este caso elegiremos Add JAR/Folder



Resultado:



1. En el proyecto **modug2** debe crear una clase y colocar el nombre **VerEmpleado**, dentro del paquete "**sv.org.arrupe.empleado**".

2. **Digitar el siguiente código:**

```java
package sv.org.arrupe.empleado;
import java.sql.*;
/**
 *
 * @author DELL
 */
public class VerEmpleado {
    public VerEmpleado() throws SQLException, ClassNotFoundException{
        //Se utilizara un try para trabajar posibles errores de MySQL
        try{
            //Obteniendo el driver para mysql
            Class.forName(className: "com.mysql.jdbc.Driver");

            //conectando con la bdd
            Connection conexion = DriverManager.getConnection(url: "jdbc:mysql://localhost/modug2",user:"root",password:"");
            //NOTA el campo "" es de la contraseña en caso de haberse creado a la hora de instalar su gestor de BDD
            //vamos a permitir ejecutar sentencias SQL sin utilizar parametros
            Statement s = conexion.createStatement();

            //Ahora vamos a obtener el resultado de la consulta SQL que hemos realizado
            ResultSet rs = s.executeQuery(string: "SELECT * FROM Empleados");
```

```java
            //Recorreremos el ResultSet mostrando por pantalla o consola los resultados

            while (rs.next()){
                //Vamos a mostrar los datos
                System.out.println("ID: "+rs.getInt(i: 1)+
                        "\nNombre: "+rs.getString(i: 2)+
                        "\nApellidos: "+rs.getString(i: 3)+
                        "\nTelefono: "+rs.getString(i: 4));
                System.out.println(x: "***************************");
            }

            //Cerrando la conexión con la base de datos NO OLVIDAR
            conexion.close();
        }
        catch(ClassNotFoundException e1){
            //Por si el drive falla
            System.out.println("ERROR: Drive no encontrado"+e1.getMessage());
        }
        catch(SQLException e2){
            //Error por si el login falla o la query es erronea
            System.out.println("ERROR: Fallo en SQL"+e2.getMessage());
        }
    }
```

```java
    //Metodo principal, instancia una clase prueba SQL
    public static void main(String[] args) throws SQLException, ClassNotFoundException{
        new VerEmpleado();
    }
}
```

3. Ejecutar la aplicación y obtendremos los siguientes resultados:

```
Notifications   Output ×
Java DB Database Process ×   SQL 1 execution ×   modug2 (run) ×
run:
Loading class `com.mysql.jdbc.Driver'. This is deprecated. The new driver class is `com.mysql.cj.jdbc.Driver'. The driver is automatically regist
ID: 1
Nombre: Katherine
Apellidos: Canales
Telefono: 2222-2222
***************************
ID: 2
Nombre: Orquidea
Apellidos: Mejia
Telefono: 2121-2121
***************************
ID: 3
Nombre: Daniel
Apellidos: Sosa
Telefono: 2324-2526
***************************
BUILD SUCCESSFUL (total time: 1 second)
Starting the Java DB Server
```

4. Forma alternativa de mostrar los datos.

Para obtener los distintos atributos de la base se utilizarán, normalmente, los métodos getString(atributo), cuyo parámetro es una cadena con el nombre del atributo que queremos recuperar.

```
//Se recorre el ResultSet, mostrando por pantalla los resultados.
while (rs.next()){
System.out.println ("ID: "+rs.getInt(string: "Codigo")+
        "\nNombre: "+rs.getString(string: "Nombres")+
        "\nApellidos: "+rs.getString(string: "Apellidos")+
        "\nTelefono: "+rs.getString(string: "Telefono"));
System.out.println(x: "***********************************");
}
```

Ingresar datos desde una aplicación a una base de datos

1. En el proyecto modug2 debe crear una clase y colocar el nombre IngresoDatos, dentro del paquete "**sv.org.arrupe.empleado**".
2. Digitar el siguiente código.

```
package sv.org.arrupe.empleado;
import static java.lang.Boolean.TRUE;
import java.sql.*;
import javax.swing.JOptionPane;
import sv.org.arrupe.util.*;
/**
 *
 * @author DELL
 */
public class IngresoDatos {
    private int id;
    private String ids;
    private String nombre;
    private String apellido;
    private String telefono;

    private Connection conexion;
    private ResultSet rs;
    private Statement s;

    public IngresoDatos(){
        //Iniciamos con un try
```

```java
    try{
        Class.forName(className: "com.mysql.jdbc.Driver");

        conexion = DriverManager.getConneotion(
        url: "jdbc:mysql://localhost/modug2",user:"root",password:"");

        s = conexion.createStatement();

        //Declaramos el metodo para ingresar los valores
        ingreso();
        s.executeUpdate("Insert into Empleados values ("+id+",\""+nombre+"\",\""+apellido+"\",\""+telefono+"\")");

        JOptionPane.showMessageDialog(parentComponent: null,message: "Persona ingresada correctamente");
    }

    catch(ClassNotFoundException e1) {
        System.out.println("ERROR:No encuentro el driver de la BD: "+e1.getMessage());
        System.exit(status: 0);
    }
    catch (SQLException e2) {
        System.out.println("ERROR:Fallo en SQL: "+e2.getMessage()); System.exit(status: 0);
    }
}
```

```java
public void ingreso(){
    ids = JOptionPane.showInputDialog(message: "Ingrese el ID");
    id = Integer.parseInt(s: ids);

    nombre=JOptionPane.showInputDialog(message: "Ingrese el Nombre");
    apellido = JOptionPane.showInputDialog(message: "Ingrese el apellido");
    telefono = JOptionPane.showInputDialog(message: "Ingrese el telefono");

    do{
        if(MatchTelephone.compareTelephone(telefono)==TRUE){
            break;
        }
        else{
            JOptionPane.showMessageDialog(parentComponent: null, message: "Numero de telefono invalido");
            telefono = JOptionPane.showInputDialog(message: "Ingrese telefono valido");
        }
    }
        while(true);
        }
```

```java
        public void mostrardatos() throws SQLException{
            rs = s.executeQuery(string: "SELECT * FROM Empleados");
            while (rs.next())
            {
                JOptionPane.showMessageDialog(parentComponent: null,"ID: "+rs.getString(string: "Codigo")+
                        "\nNombre: "+rs.getString(string: "Nombres")+
                        "\nApellidos: "+rs.getString(string: "Apellidos")+
                        "\nTelefono: "+rs.getString(string: "Telefono"));
            }
        }
        public void cierreconexion() throws SQLException{
            if (conexion != null){
                conexion.close();
            }
        }
        public static void main(String[] args)  throws SQLException {
            IngresoDatos ing = new IngresoDatos();

        ing.mostrardatos();
        ing.cierreconexion();
        }
}
```

3. Crear la clase llamada **MatchTelephone** dentro del paquete "**sv.org.arrupe.util**" y agregar el siguiente código, esta clase utilizara expresiones regulares para validar el formato de un número de teléfono.

```java
package sv.org.arrupe.util;

import static java.lang.Boolean.FALSE;
import static java.lang.Boolean.TRUE;
import java.util.regex.Matcher;
import java.util.regex.Pattern;

/**
 *
 * @author DELL
 */
public class MatchTelephone {
  public static void main(String[] args) {
      compareTelephone(telefono:"7123-4444");
  }
  public static boolean compareTelephone (String telefono) {
      String expresion="(2|7)\\d{3}-\\d{4}";
      Pattern pat = Pattern.compile(regex: expresion);
      Matcher mat = pat.matcher(input: telefono);
      if (mat.matches()) {
          System.out.println(x: "SI");
          return TRUE;
      }
      return FALSE;
  } //Cierre del main
}
```

4. Para este ejemplo se realizara una análisis que nos permita comprender que no es lo mismo manejar datos “Vacios o Nulos”, en primer lugar crearemos la clase **Conexión** para que este en un paquete diferente “**sv.org.arrupe.util**” y solo la invoquemos para futuros desarrollos.

```java
package sv.org.arrupe.util;
import java.sql.*;
/**
 *
 * @author DELL
 */
public class conexion {
    private Connection conexion =null;
    private Statement s =null;
    private ResultSet rs=null;
    private String ingresoempleados="";
//Constructor
    public conexion() throws SQLException{
        try
        {
            //obtenemos el driver de para mysql
            Class.forName(className: "com.mysql.jdbc.Driver");
            // Se obtiene una conexión con la base de datos.
            conexion = DriverManager.getConnection (
                url: "jdbc:mysql://localhost/guia4",user:"root", password:"");
            // Permite ejecutar sentencias SQL sin parámetros
            s = conexion.createStatement();
            System.out.println(x: "Conexion Exitosa");
        }
        catch (ClassNotFoundException e1) {
            //Error si no puedo leer el driver de MySQL
            System.out.println("ERROR:No encuentro el driver de la BD: "+e1.getMessage());
        }
    }
```

```java
    //Metodo que permite fijar la tabla resultado de la pregunta
    //SQL realizada
    public void setRs(String sql)  {
        try {
            this.rs = s.executeQuery(string: sql);
        }
        catch (SQLException e2) {
            System.out.println("ERROR:Fallo en SQL: "+e2.getMessage());
        }
    }

    //Metodo que recibe un sql como parametro que sea un update,insert.delete
    public void setQuery(String sql) throws SQLException {
        this.s.executeUpdate(string: sql);
    }

    //Metodo que cierra la conexion
    public void cerrarConexion() throws SQLException{
        conexion.close();
    }

}
```

5. Crear la clase llamada "**InsertNulos**" dentro del paquete "**sv.org.arrupe.nulos**" y agregar el siguiente código.

```java
package sv.org.arrupe.nulos;

import java.sql.ResultSet;
import java.sql.SQLException;
import sv.org.arrupe.util.conexion;

/**
 *
 * @author DELL
 */
public class InsertNulos {
    public static void main(String[] args) throws SQLException {

        conexion con = new conexion();
        //String sql = "insert into empleados values(7,'','Castillo',null)";
        //con.setQuery(sql);

        String sql = "select nombres from empleados ";
        ResultSet rs ;

        con.setRs(sql);
        rs= con.getRs();

        String nombre;

        while (rs.next()){
            nombre=rs.getString(i: 1);

            if (nombre == null){
                System.out.println("Nombre 'Null': " + nombre);
            }else if(nombre.equals(anObject:"")){
```

```java
                System.out.println("Nombre Vacio: " + nombre);
            }else{
                System.out.println("Nombre Con Datos: " + nombre);
            }


        }
        con.cerrarConexion();
    }

}
```

6. Ejecutar un par de sentencias update para la tabla **empleados.**
   - insert into empleados values(5,'','Castillo',null);
   - insert into empleados values(6,null,Lopez,null);

7. Ejecutar la clase **InsertNulos** para ver un resultado como el de la siguiente imagen

```
run:
Loading class `com.mysql.jdbc.Driver'. This is
Conexion Exitosa
Nombre Con Datos: Katherine
Nombre Con Datos: Orquidea
Nombre Con Datos: Daniel
Nombre Con Datos: Jimena
Nombre Con Datos: Daniel
Nombre Vacio:
BUILD SUCCESSFUL (total time: 0 seconds)
```

ACTIVIDAD COMPLEMENTARIA

- **Para poder realizar este ejercicio complementario deberá crear la base de datos "EstudiantesArrupe".**

Tabla alumno

Nombre Columna	Tipo de dato
cod_alumno	int primary key
Nombre	varchar(80)
Apellido	varchar(80)
Edad	int
Direccion	varchar(100)

Tabla materia

Nombre Columna	Tipo de dato
cod_materia	int primary key
Nombre	varchar(25)
Descripción	varchar(100)

Tabla alumno_materia

Nombre Columna	Tipo de dato
cod_alumno	int (llave foránea)
cod_materia	int (llave foránea)

Crear el CRUD a las 3 tablas anteriores:

- Alumno
- Materia
- Alumno_materia

Reportes

1. Crear una aplicación donde muestre la información de cada una de las tablas anteriores, para ello crear tres métodos diferentes que serán invocados según la necesidad.
2. Mostrar las materias que está cursando un alumno específico. T

Nota: Para cada ejercicio solicitado utilizar JoptionPane y validar los datos. Debe usar una clase pre-fabricada para validar.

BIBLIOGRAFÍA

- Oracle. (s. f.). *Java downloads*. Oracle.com. Recuperado el 4 de marzo de 2024, de https://www.oracle.com/java/technologies/downloads/
- Apache NetBeans. (s. f.). *Apache NetBeans releases*. Apache.org. Recuperado el 4 de marzo de 2024, de https://netbeans.apache.org/front/main/download/index.html
- Rodríguez, & Walter Sagástegui, A. (s. f.). *Configurar Java en Windows. Variables de entorno JAVA_HOME y PATH (CU00610B)*. Aprenderaprogramar.com. Recuperado el 4 de marzo de 2024, de https://www.aprenderaprogramar.com/index.php?option=com_content&view=article&id=389:configurar-java-en-windows-variables-de-entorno-javahome-y-path-cu00610b&catid=68&Itemid=188